--open SCRIPT_NAME, which starts your script as a background process. Then in a different terminal you run rdbg --attach to connect to the running process in a debugger session.

Alternately, you can use require debug/open in the background process, which allows you to skip the rdbg --open step and go straight to the attach step. Doing so stops the program on the first line; if you want the program to run normally, use require debug/open_nonstop. However, don't do that in production—it'll leave the debug socket open waiting for a debugger to attach.

Here's a short sample program that we'll use to demonstrate the Ruby debugger:

debugging/debugger_test.rb

```
require "debug"

class CashRegister
  attr_accessor :tax_rate

  def initialize(tax_rate)
    @tax_rate = tax_rate
    @items = []
  end

  def add_item(item, quantity, price_in_cents)
    @items << {item:, quantity:, price_in_cents:}
  end

  def subtotals
    @items.map {_1[:quantity] * _1[:price_in_cents]}
  end

  def pre_tax_total = subtotals.sum

  def tax = pre_tax_total * tax_rate

  def total = pre_tax_total + tax

  def sale_price(price_in_cents, discount)
    price_in_cents * (1.0 - discount)
  end
end

register = CashRegister.new(0.05)
register.add_item("pen", 3, 499)
binding.break
register.add_item("paper", 2, register.sale_price(799, 25))
p register.total
```

We don't think there's a bug in here, but let's trace through it via the debugger.

When you enter the debugging tool via either rdbg or binding.break, you are taken to the command-line interface for the tool, which looks like this:

```
[27, 34] in code/debugger/debugger_test.rb
    27|   end
    28| end
    29|
    30| register = CashRegister.new(0.05)
    31| register.add_item("pen", 3, 499)
=>  32| binding.break
    33| register.add_item("paper", 2, register.sale_price(799, 25))
```